

NFE204 – Bases de données documentaires et distribuées

Etude du système Neo4j

14/05/2020

SOMMAIRE

Sommaire

OBJET DU DOCUMENT	2
REFERENCES	2

PARTIE I : THEORIE

INTRODUCTION	3
PRINCIPALES CARACTERISTIQUES	4
MODELE DE DONNEES	5
STOCKAGE ET INDEX	6
ARCHITECTURES REPARTIES.....	7
Cluster haute disponibilité (HA).....	7
Cluster causal	9
SHARDING / PARTITIONNEMENT	11
INTEGRATION	12
SECURITE DES ACCES.....	14
SAUVEGARDES	14
LANGAGE D'INTERROGATION (CYPHER).....	15
Motifs	15
CRUD	15
Index.....	16
Contraintes.....	16
IMPORTATION DES DONNEES.....	17
MONITORING	18

PARTIE II : PRATIQUE

INSTALLATION	19
CLIENTS	20
JEU DE DONNEES.....	20
MODELE DE DONNEES	21
IMPORTATION DE DONNEES.....	21
INTERROGATION DE LA BASE DE DONNEES.....	23
MISE EN OEUVRE CLUSTER CAUSAL.....	26

ANNEXES

OBJET DU DOCUMENT

Ce document a pour objet l'étude du système de gestion de base de données « Neo4j ». Il est composé de deux chapitres :

- Un premier chapitre « théorique » qui présente les principales caractéristiques de ce système NoSQL (modèle de données, stockage, passage à l'échelle, applications)
- Un deuxième chapitre « pratique » qui décrit sous la forme d'un tutoriel plusieurs opérations pour la mise en œuvre et l'exploitation de ce système (installation, imports de données, recherches, modifications, résilience, scalabilité etc.).

REFERENCES

La documentation officielle sur le site internet : <http://neo4j.com/docs>

- | | |
|--|-------------------------------------|
| [1] neo4j-getting-started-4.0.pdf | Guide de démarrage |
| [2] neo4j-cypher-manual-4.0.pdf | Manuel du langage Cypher |
| [3] cypher-refcard-4.0.pdf | Carte référence du langage Cypher |
| [4] neo4j-operations-manual-4.0.pdf | Comment déployer et maintenir Neo4j |
| [5] neo4j-http-api-4.0.pdf | API transactionnelle http |

PARTIE I : THEORIE

INTRODUCTION

Neo4j est un système de gestion de bases de données orienté graphe. Son modèle mathématique n'est pas fondé sur l'algèbre relationnelle mais sur la théorie des graphes, ainsi il est classé parmi les systèmes NoSQL.

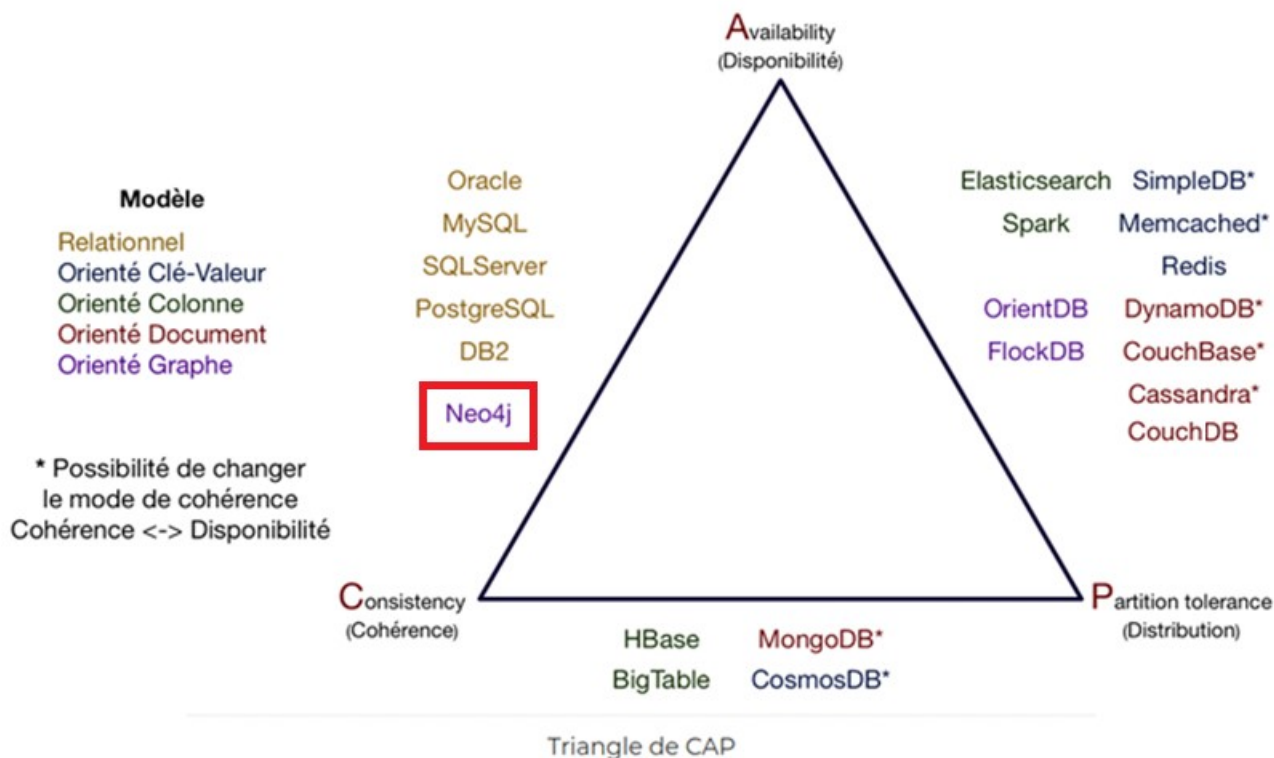
Neo4j est particulièrement adapté dans des contextes où **les données sont fortement connectées** (nombreux liens) et organisées selon des **modèles complexes et flexibles** (sans schéma prédéfini, construit dynamiquement). En outre, il présente des **performances exceptionnelles** (temps réel) en termes de lecture et de parcours de données dans un graphe. Il simplifie **le développement** en intégrant une API REST et des algorithmes classiques de la théorie des graphes (plus court chemin, détection de boucles, identification de sous-graphes, calcul de centralité...)

Les bases de données orientées graphe sont conçues pour la résolution de problèmes spécifiques, pour lesquels les classiques bases de données relationnelles (SGBDR) et même d'autres systèmes NoSql s'avèrent complètement inefficaces. On peut citer notamment les multiples jointures très coûteuses en performances, le schéma rigide et le passage à l'échelle difficile pour les SGBDR. Et concernant les bases NoSQL (orienté document ou clé-valeur), la plupart ne supportent pas l'intégrité transactionnelle (ACID) ni le concept de relation entre les objets (documents).

Les **cas d'usage** des bases de données orientées graphe sont nombreux : dans l'énergie, les services financiers (pour la détection de fraudes), la gestion des ressources humaines (réseaux sociaux), l'informatique & télécommunications, les jeux, la logistique (pour le routage de colis), mais aussi la distribution (pour la recommandation de produits) et le transport (pour le calcul d'itinéraires).

Parmi les grands noms, on citera : Viadeo, ebay, National Geographic, Adobe, Cisco, Deutsche Telekom, Walmart, Glowbl, Career Builder, Meetic, SFR, Voyages-SNCF, etc.

Positionnement de Neo4j dans le triangle de CAP



PRINCIPALES CARACTERISTIQUES

Name	: 
Catégorie	: NoSql orientée graphe
Société	: Neo Technology (Suède)
Site web	: neo4j.com
Documentation	: neo4j.com/docs
License	: Open Source Java - GPLv3 Community, AGPLv3 Entreprise (assistance et outils)
Système d'exploitation	: multi-plateformes (Linux, OS X, Solaris, Windows)
Première version	: 2010
Version actuelle	: 4.0.2, March 2020
Langage d'interrogation	: Cypher (simple et efficace), initiative de normalisation avec OpenCypher (langage commun pour les graphes)
APIs	: Bolt protocol, Cypher query language, Java API, Neo4j-OGM, RESTful HTTP API, Spring Data Neo4j, TinkerPop 3
Intégration - Langages	: .Net, Java, Scala, Javascript, Perl, Php, Python ...
Intégration - ETL	: import de données en masse
Procédures stockées	: oui
Triggers	: oui
Partitionnement	: aucun
Réplication	: version entreprise uniquement – disponibilité élevée avec cluster
Volumétrie	: des milliards de nœuds et de relations
MapReduce	: non
Schéma pré-établi	: non (sauf contraintes d'unicité)
Intégrité référentielle	: oui (la suppression d'un nœud supprime aussi toutes les relations))
Transactionnel ACID	: oui
Accès concurrents	: oui
Persistance	: oui
Confidentialité	: utilisateur, rôles, permissions. Authentification (LDAP, Active Directory, Kerberos)

Volumétrie

	Maximum
Bases de données hébergées	1
Nombre de nœuds	2^{35}
Nombre de relations	2^{35}
Nombre de propriétés	2^{36}
Nombre de label/clusters	2 milliards
Nombre d'entrées par label/cluster	2^{35}
Nombre d'index	
Taille pour une entrée	1To
Nombre de réponses à une requête	2 milliards
Quantité de données modifiables dans une transaction	Espace libre en mémoire

MODELE DE DONNEES

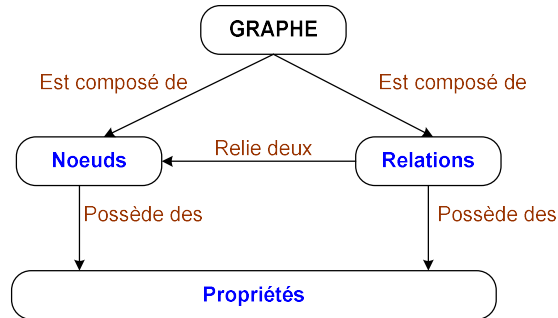
On distingue essentiellement deux types d'entités persistées:

le nœud : équivalent à une ligne dans une table d'un SGBDR

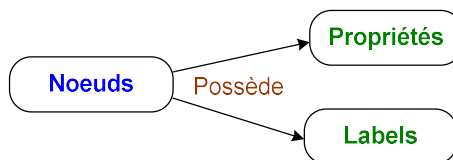
la relation (ou arc) : équivalent à un lien entre deux lignes d'une même table ou de tables différentes

La structure d'une entité (nœud ou relation) est définie au moment de son stockage (et non préalablement comme les tables d'une base de données relationnelle), le schéma est donc dynamique.

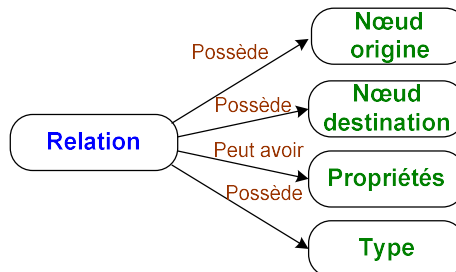
Le modèle de donnée Neo4j est très simple et se résume par le diagramme suivant :



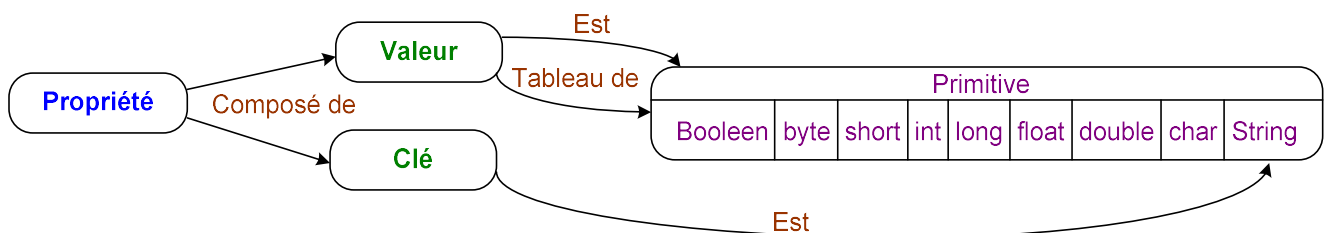
Pour différencier les nœuds, Neo4j apporte la notion de label. Ceux-ci permettent de donner un rôle ou un type à un nœud (un nœud peut avoir plusieurs labels).



Les relations sont constituées d'un nœud de départ, d'un nœud d'arrivée (avec un sens) et d'un type.

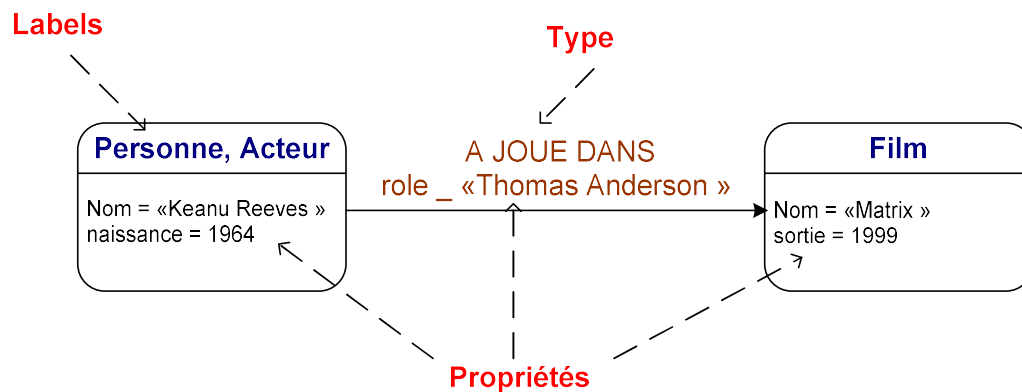


Les nœuds et les relations sont des enregistrements composés de propriétés sous la forme de paires clef/valeur, sans schéma préétabli. Les propriétés peuvent donc être différentes d'un nœud ou d'une relation à l'autre. Les propriétés sont typées, les types possibles sont les types primitifs de Java, ou un tableau de type primitif.



Les métadonnées des nœuds (propriétés, labels) et des relations (propriétés, type) sont utiles pour le parcours du graphe, elles permettent de comparer leur valeur avec le motif de la requête.

Exemple :



Correspondance entre une base de données relationnelle et une base de donnée orientée graphe :

SGBDR	Neo4j
Table	Sous graphe
Ligne	Noeud
Colonne	Propriété (clé) avec sa valeur
Contrainte	Relation
Joins	Parcours du graphe, visite des noeuds

STOCKAGE ET INDEX

Avec un objectif de performance et de scalabilité, les SGBDR ainsi que d'autres bases orientées document ne sont pas adaptées au stockage d'un graphe. Les structures de stockage doivent être dédiées aux nœuds et relations.

Les performances concernant le parcours du graphe sont primordiales. Un nœud de départ est choisi et le parcours du graphe se fait à partir de là (principe du graph traversal). Le système de stockage est conçu de telle sorte que chaque voisin d'un nœud est accessible par un pointeur physique, ce qui évite d'avoir recours à un index.

Pour rechercher les données, le parcours du graphe est alors une combinaison de cheminement indiqué par des pointeurs (*chasse au pointeur*) et de recherche de correspondance avec un motif (*pattern matching*).

Les performances ne dépendent pas de la taille totale de l'ensemble des données mais uniquement du sous-graphe constitué uniquement des données interrogées. Cela conduit à des temps de performance à peu près constant, liés à la taille de l'ensemble des résultats (Environ 200 parcours de relation par milliseconde).

Bien que les opérations de traversée des relations soient l'un des points forts de Neo4j, on a souvent besoin aussi de fonctionnalités avancées de recherche d'information. La recherche *plein texte* des propriétés dans tout le graphe est un exemple typique. Neo4j utilise un système d'index externe avec une bibliothèque intégrée de Lucene.

Lucene est un exemple d'index externe du graphe. Cependant, avec un moteur de graphe rapide il y a plusieurs stratégies pour construire des structures d'indexation dans le graphe lui-même, réduisant les schémas de parcours pour des jeux de données et domaines spécifiques. Par exemple, il existe les timeline et les B-Arbres pour les données à 1 dimension, les RTrees et Quadrees pour indexer les données à 2 dimensions (SIG et modélisation Spatiale) et bien d'autres. Souvent un autre schéma utile est de connecter des sous-graphes importants directement au nœud racine de manière à avoir des raccourcis sur les nœuds de départ importants.

Le principal intérêt à utiliser des index dans un graphe est de trouver rapidement le nœud de départ dans le parcours du graphe. Avec Neo4J, il est possible de créer des index d'un label de nœuds à partir de Cypher, de l'interface REST ou d'une extension Java.

ARCHITECTURES REPARTIES

La version Entreprise de Neo4j fournit deux solutions d'architecture pour la scalabilité (augmentation du nombre de clients et par conséquent du nombre de requêtes et donc du nombre d'opérations sur le graphe), la haute disponibilité (continuité de service, capacité du système à répondre aux demandes des clients dans un délai acceptable) et la tolérance aux pannes (résilience ou failover) :

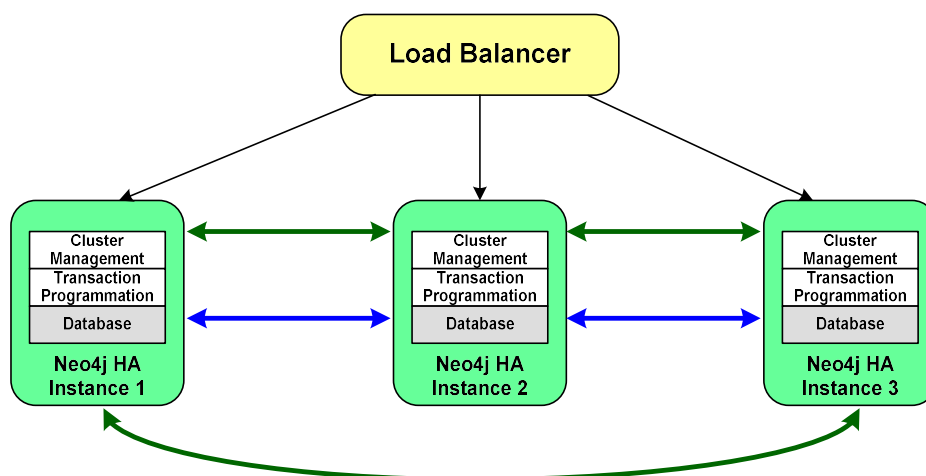
- la première solution appelée « cluster haute disponibilité (HA) » est simple à mettre en œuvre mais ne permet pas une cohérence forte.
- la deuxième solution appelée « cluster causal » garanti une cohérence forte en respectant les propriétés des transactions ACID.

La différence entre ces deux architectures porte essentiellement sur la cohérence, il faut que les mêmes données soient en permanence accessibles depuis n'importe quel serveur ; toute lecture intervenant juste après une écriture doit tenir compte de la dernière mise à jour des données.

L'enjeu de ces architectures réparties est donc de maintenir des niveaux de disponibilité et de performance du système malgré les aléas de pannes tout en assurant en permanence la cohérence des données. Elles doivent en plus être évolutives en permettant une augmentation de la charge.

Les mécanismes mis en œuvre sont la réplication des données, une surveillance et synchronisation mutuelle, une répartition de la charge, la gestion d'une mémoire cache de manière à minimiser les accès disque (coûteux en temps).

Cluster haute disponibilité (HA)



C'est une solution du type cluster maître-esclave avec les caractéristiques suivantes:

Chaque serveur possède la même pile logicielle, le même jeu de données et peut répondre à toutes les requêtes des clients. Sur une machine on a généralement un seul graphe. Cette structure garantit une réplique complète du graphe sur tous les serveurs. On a ainsi plusieurs instances du même graphe dans un seul cluster.

Tous les nœuds sont identiques mais se distinguent par les rôles suivant:

- **instance maître** : instance responsable de la réplication des données sur les nœuds esclaves de manière asynchrone (gestion de la propagation des transactions);
- **instance esclave** : instance accessible de préférence pour les opérations en lecture (Toutefois il serait possible de la solliciter pour des opérations en écriture) ;
- **instance arbitre** : instance esclave n'exposant aucune donnée (pas de réplication des données) mais qui participe à la gestion du cluster (élection d'un nouveau maître).

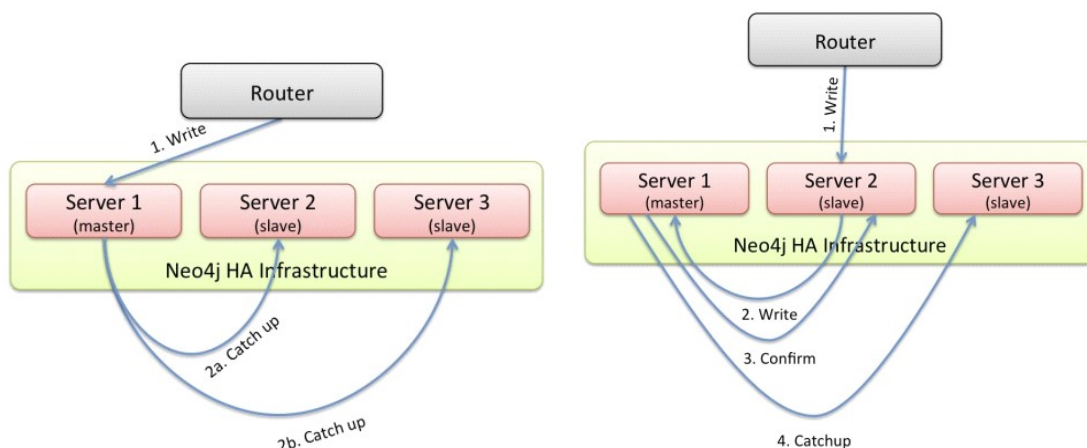
Tous les nœuds ont un identifiant unique (un entier) défini dans le fichier de configuration de Neo4j (neo4j.conf).

Un seul maître est actif, il est élu au démarrage de la base de données. Si une panne survient sur le maître, le gestionnaire de cluster réparti sur tous les nœuds va élire un nouveau maître suivant un algorithme basé sur Paxos. Pour que cela soit possible il faut que plus de 50 % des membres de ce cluster soient encore présents (quorum). Selon le nombre de maîtres défectueux que l'on souhaite supporter (F), un cluster devra comporter $M=2F+1$ instances (ex : si on tolère deux pannes de maîtres, $M = 5$ serveurs doivent être déployés). C'est l'esclave avec l'identifiant de transaction le plus récent (celui qui contient les dernières données) qui sera élu. Si plusieurs esclaves ont les mêmes données alors on choisira celui qui possède l'identifiant d'instance le plus bas.

Lors de l'ajout d'un nœud dans un cluster, il se resynchronise avec le maître en écrasant son ancienne base puis il devient un esclave.

Le routeur (load balancer) peut être configuré pour rediriger les requêtes vers un membre du cluster, suivant différents critères (par exemple en distinguant les requêtes d'écriture des requêtes de lecture, suivant la « région » d'où elles sont émises par les clients, ou suivant l'application). Cela permet à Neo4j d'optimiser le contenu de sa mémoire cache distribuée (cache sharding) sur les serveurs du cluster. Les données stockées dans des fichiers sont identiques sur tous les serveurs, cependant dans la mémoire (le cache) les parties du graphe chargées sont différentes d'un serveur à l'autre. Ce principe de mémoire cache est utilisé dans la plupart des systèmes de gestion de base de données, il permet globalement de répondre plus rapidement aux requêtes.

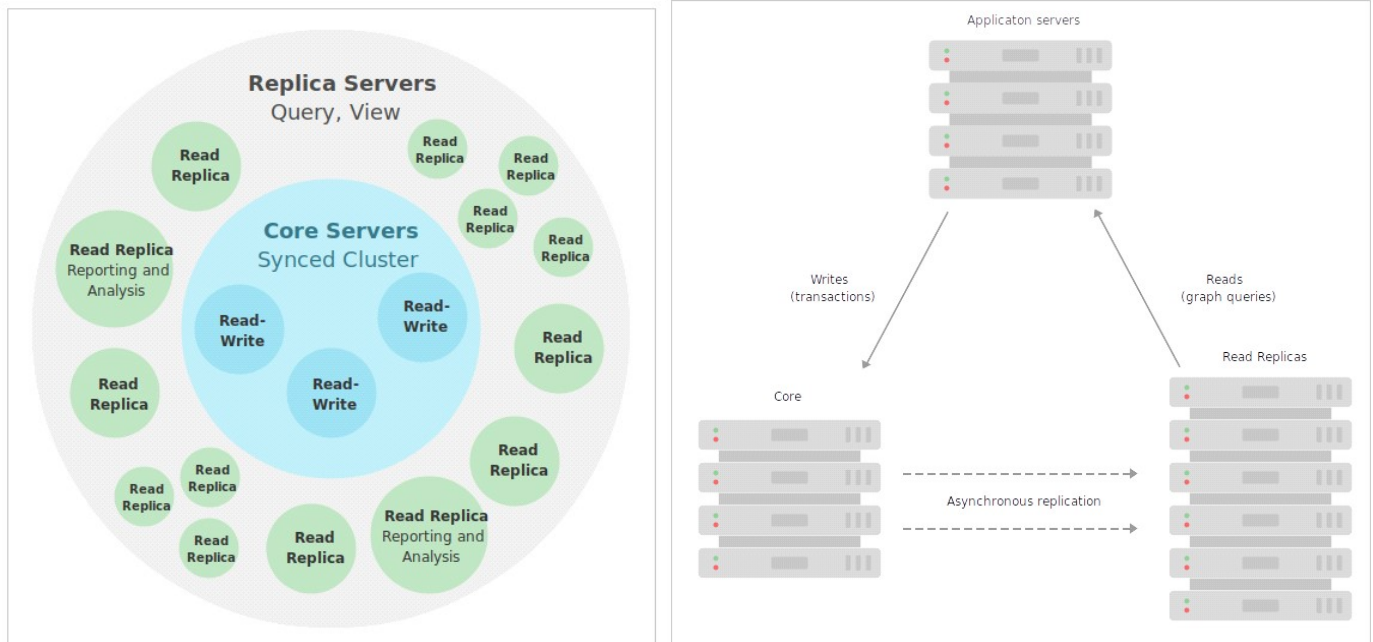
Les écritures sont répliquées à partir du maître vers tous les esclaves à intervalles fréquents. L'usage de transactions est obligatoire pour bénéficier des propriétés ACID et ainsi assurer la cohérence des graphes à terme. Toute demande d'écriture faite à un esclave est redirigée vers le maître.



Les lectures ou recherches s'effectuent sur n'importe quel serveur, ce qui multiplie donc la puissance globale de calcul.

Un ou plusieurs répartiteurs de charge (load balancer) peuvent être utilisés comme HAProxy, NGINX or Elastic Load Balancer (Amazon). Ces répartiteurs sont configurables, il est possible de définir une stratégie de routage qui prend en compte le failover.

Cluster causal



Avec le cluster HA, il y a plusieurs inconvénients :

- A tout moment, une seule instance est responsable des transactions. Ainsi dans le cas où le maître tombe en panne et que les esclaves n'ont pas eu le temps de se synchroniser, il peut y avoir une perte de données ;
- Il peut y avoir des situations où la cohérence à terme ne convient pas.

Le cluster causal va améliorer ces faiblesses mais avec un impact sur la latence.

Le cluster causal fournit trois fonctionnalités principales :

- disponibilité : les serveurs primaires (core) fournissent une plate-forme tolérante aux pannes pour le traitement des transactions. Ils restent disponibles en écriture tant qu'une majorité simple de ces serveurs fonctionne ;
- scalabilité : les serveurs secondaires (replica) (lecture seule) offrent la possibilité d'élargir le cluster pour traiter des requêtes en lecture qui impliquent un fort volume de données ou un traitement complexe dans un large graphe ;
- cohérence causale : garantit qu'une application cliente puisse lire au moins ses propres écritures.

On distingue deux sous-ensembles de nœuds :

- Les read-write core constituent le noyau du cluster. Cette partie gère l'ensemble du cluster et assure entièrement la sauvegarde des données, la disponibilité et la cohérence causale du système.
- Les read replica sont optionnels et n'interviennent pas dans la gestion du cluster contrairement aux instances esclaves du cluster HA. Leur rôle consiste à décharger les serveurs core du travail de lecture seule (et du traitement de ce type de requêtes). La perte d'une instance read-replica n'a aucun impact sur la disponibilité et la tolérance aux pannes. Ils sont en quelques sortes un cache pour les données. Cette partie permet la scalabilité du système tout en maintenant les performances.

Les transactions se produisent au niveau des serveurs core. Pour qu'une transaction soit valide (acquittée), il faut qu'elle soit acceptée par une majorité de serveurs core du cluster ($\text{Nombre d'instances}/2+1$). S'il n'y a plus de majorité disponible, alors le graphe ne sera accessible qu'en lecture seule.

Plus il y a de serveurs core, plus la durée du traitement des transactions augmente, cela renforce l'intérêt de libérer ces serveurs du travail de lecture. Aussi, on choisit un nombre minimal de serveurs satisfaisant une tolérance de panne prédéfinie. Le nombre total de serveurs core M permettant d'absorber F défaillances est alors de $M = 2F + 1$. Un noyau avec seulement deux serveurs ne sera pas tolérant aux pannes, si un de ces deux s'arrête, celui qui reste passera en mode lecture seule.

Les réplicas sont mis à jour de manière asynchrone. Ils demandent périodiquement à un nœud core les transactions qu'ils n'ont pas encore traité.

Avec ce type d'architecture, le routeur joue un rôle essentiel, il redirige les requêtes d'écriture vers le core et les requêtes de lecture vers les réplicas.

Le driver associé au protocole bolt-routing a une image de la topologie du cluster afin d'effectuer le routage qui convient en fonction de la disponibilité des instances et du type de transaction à effectuer (lecture ou écriture).

Le principe de cohérence causale consiste à obtenir une lecture des données juste. Il faut que chacune des transactions liées aux opérations d'écriture (la cause) dans lesquelles les données lues sont impliquées soit effective sur l'instance concernée par la lecture.

Le routeur va effectuer une écriture en posant un signet sur la transaction (bookmark) : il s'agit d'un identifiant qui va servir de repère au système. Cela fait, il va ensuite chercher à lire les données correspondant à ce signet de transaction. Rien ne garantit que l'instance Neo4j qui sera appelée possède déjà ces données. Elle va donc consulter ses journaux de transactions et, si la transaction du signet n'est pas encore passée, elle va se mettre en attente jusqu'à ce qu'elle soit traitée. Cela permet de garantir que l'application puisse lire au moins ses propres écritures.

Le fonctionnement du cluster causal est régi par quatre mécanismes ou protocoles :

- protocole de découverte (Discovery protocol): chaque nouveau nœud prend connaissance de la topologie du cluster (liste des autres membres)
- protocole de transaction (Raft protocol): algorithme de consensus qui synchronise les journaux des transactions du cluster. Chaque transaction est enregistrée dans un journal répliqué sur chaque serveur core. Ce protocole garantit l'ordre d'arrivée des entrées dans le journal sur chacun des serveurs, ainsi les transactions sont toujours effectuées selon une même séquence sur tous les serveurs. Parmi les nœuds core, l'un d'entre eux a le rôle de leader (Maître) et les autres de suiveurs (esclaves). Le leader détermine l'ordonnancement des transactions. La perte d'un leader entraîne une élection pour en désigner un nouveau (celui dont le journal est le plus complet), cette recatrisation dure environ 500ms durant lesquelles une exception est remontée au client si une écriture est demandée. Le client doit alors refaire une tentative.
- Protocole de rattrapage (Catchup protocol): lorsqu'un nouveau serveur vient de rejoindre le cluster, il doit répliquer la base de données et le journal de transactions. Ensuite, les serveurs réplica viennent mettre à jour périodiquement leur journal et exécuter les transactions.
- Protocole de sauvegarde (backup protocol) : sauvegarde périodique à chaud des données d'une instance du cluster sur un espace de stockage tiers. Deux types de sauvegarde sont possibles : base complète ou différentielle (incrémentale).

Il est à noter, selon le principe du cluster causal, il est également possible de mettre en place une solution répartie sur plusieurs data-center.

SHARDING / PARTITIONNEMENT

Neo4j ne sait pas partitionner un graphe sur plusieurs machines de sorte que l'accès en lecture et en écriture sur le graphe puisse être scalable horizontalement (problème NP-difficile, impossible à résoudre). Cela peut se faire dans la mesure du possible (selon le graphe) au niveau de l'application.

S'il faut dépasser la capacité d'un cluster, nous pouvons partitionner le graphe sur plusieurs instances de la base via la construction d'une logique de sharding dans l'application. Le sharding implique l'utilisation d'un identifiant synthétique pour la jointure des données sur plusieurs instances de la base au niveau applicatif. Cela dépend beaucoup de la forme du graphe, de ses frontières.

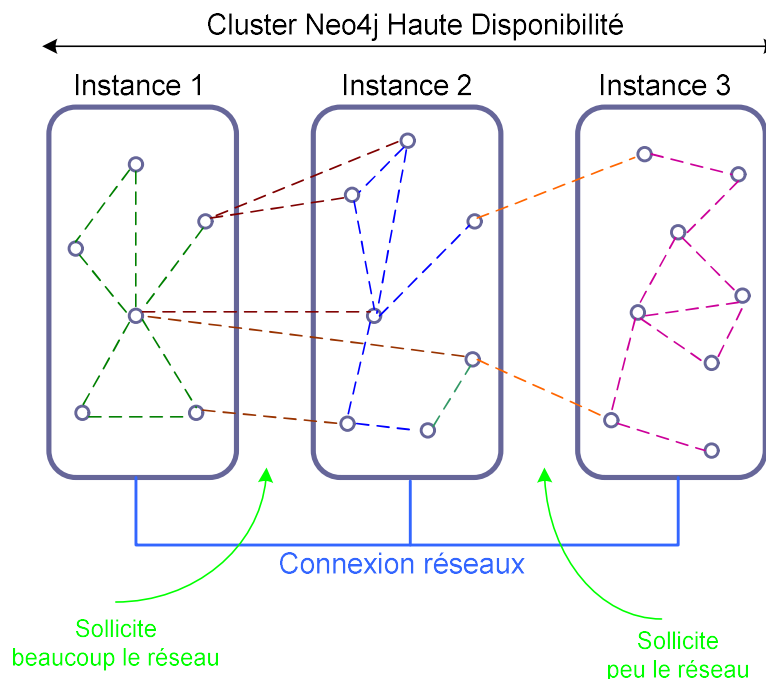
- 1) Découper le graphe en fragment en fonction d'une clé.
- 2) Placer chaque fragment sur un serveur (ou un cluster).
- 3) Maintenir un répertoire indiquant que telle clé se trouve dans tel fragment sur tel serveur.

Ce mode de partitionnement permet aussi de répartir la charge (load balancing). Il doit être dynamique (ajout/retrait de serveur transparent) pour s'adapter « élastiquement ». Il s'applique bien à des collections de paire clé-valeur.

L'application joue alors le rôle de routeur (le routeur reçoit les requêtes et détermine le(s) serveur(s) à solliciter), et doit s'inspirer de l'état de l'art pour accomplir cette mission :

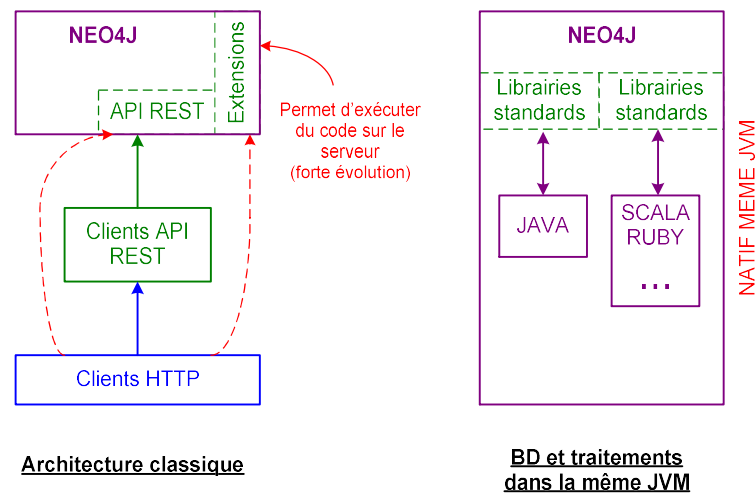
- partitionnement par intervalle (MongoDB, Cassandra, Redis). L'index est tout petit par rapport à la collection.
- partitionnement par hachage (memcached, dynamo). Faire du hachage cohérent.

Il faut en général minimiser les communications sur le réseau :



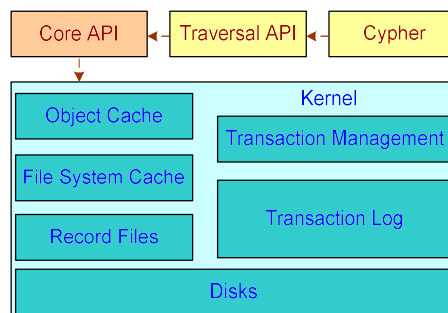
INTEGRATION

Il existe plusieurs manières d'intégrer Neo4j dans une application ou dans un système hétérogène, composé de modules qui interagissent, chacun avec une responsabilité bien définie. Il y a deux possibilités de mise en œuvre : 'embedded' ou 'server' :



Les extensions permettent d'exécuter du code sur le serveur ($\Leftrightarrow$ procédures stockées). Cependant, ces extensions sont soumises à de fortes évolutions et investir de manière pérenne peut poser problème. L'idéal reste de séparer les concepts et d'utiliser les API REST avec les différents pilotes proposés pour de nombreux langages.

Architecture logique



La version Enterprise est un package de classes qui viennent hériter des classes de base (ci-dessus), pour étendre les fonctionnalités (cluster management, transaction propagation).

Core API: API Java qui expose les primitives de graphe (nœuds, relations, propriétés et libellés).

Traversal(parcours de graphe) : processus qui visite les nœuds suivant les relations et une méthode particulière. Avec Cypher, le mode de parcours du graphe (en profondeur, en largeur) est totalement délégué alors que qu'avec un langage impératif, il y a possibilité de spécifier ce mode.

Cypher: Langage de requête qui permet d'interroger le graphe pour obtenir toute sorte d'informations.

Mécanisme de cache: Neo4j Enterprise possède deux niveaux de caches. Le premier est logique, il gère les entités (nœuds et relations) alors que le second est un cache physique qui stocke des données telles qu'elles sont écrites dans les fichiers (swap). Les caches de Neo4j ne se vident pas lors des redémarrages et il n'existe pas de commande pour les vider manuellement. Il est par contre possible de les régler très finement.

Record Files: Les fichiers sont stockés dans un répertoire par défaut. Il y a un fichier pour les nœuds, un fichier pour les relations, un autre fichier pour les paires clé/valeur des propriétés et enfin un fichier pour les index sur les labels.

Mécanisme de transactions: Neo4j garantit que les transactions sont ACID (atomicité, cohérence, isolation et durabilité). Les transactions sont historisées dans un journal permettant de maintenir la cohérence de la base de données.

Fonctions et procédures CYPHER

Une fonction retourne une valeur ou peut être un prédicat. Les procédures sont des traitements plus lourds, éventuellement asynchrone et retournent un flux de résultats. Ces fonctions et procédures s'exécutent sur le serveur (⇔ procédures stockées des SGBDR), elles simplifient le code de certaines requêtes et améliorent l'efficacité en s'exécutant au plus proche des données. Elles sont écrites en java ou scala, certaines sont intégrées de base d'autres peuvent être ajoutées par l'installation d'une extension (plugin).

Pour installer un plugin, il suffit de placer le fichier compilé .jar dans le sous-dossier /plugin puis de redémarrer le serveur en vérifiant qu'aucune erreur ne soit inscrite dans les fichiers log.

Appel d'une procédure

```
CALL nom.de.procedure
CALL nom.de.procedure() YIELD propriete1[,proprietien]
```

L'espace de noms est hiérarchique, on utilise le point comme séparateur.

dbms.* sont les procédures de gestion du système

db.* sont les procédures de gestion du graphe

Le mot clé YIELD produit le résultat, il permet de spécifier le format, i.e. la liste des noms de champs retournés.

La liste des procédures peut être obtenue par l'appel de la procédure système suivante :

```
CALL dbms.procedures()
```

Pour retourner le métagraphe, il suffit d'appeler la procédure suivante :

```
CALL db.schema()
```

Exemples :

```
CALL dbms.procedures() YIELD name
RETURN count(name) AS `Nombre de procédures CYPHER disponibles`

CALL db.labels() YIELD label
RETURN count(label) AS `Nombre de labels`, COLLECT(label) AS Labels
```

FONCTIONS ET PROCEDURE APOC

Il s'agit d'une contribution fournie par la communauté Neo4j, d'un grand nombre, très utile, de procédures et fonctions qui étendent CYPHER. Pour l'installer, il suffit de copier le fichier apoc-[version].jar dans le dossier /plugins, il faut récupérer une version qui correspond à la version Neo4j. Ces outils permettent :

- la refactorisation du graphe (ce qui est utile en phase de conception)
- la gestion de l'expiration des nœuds par configuration d'un TTL (Time To Live). Ex : le panier d'un utilisateur qui n'a pas abouti à une commande.
- L'interaction avec un SGBDR (connecteur JDBC)
- L'interaction avec Elasticsearch (lancer une requête ES ou injecter des données dans le graphe)
- L'interaction avec MongoDB, CouchDB
- Importer des données au format json ou xml
- La géolocalisation en utilisant le service openstreetmap pour retourner la longitude et la latitude d'une adresse transmise en paramètre
- L'algorithmie de graphe
- Les traitements asynchrones

API REST :

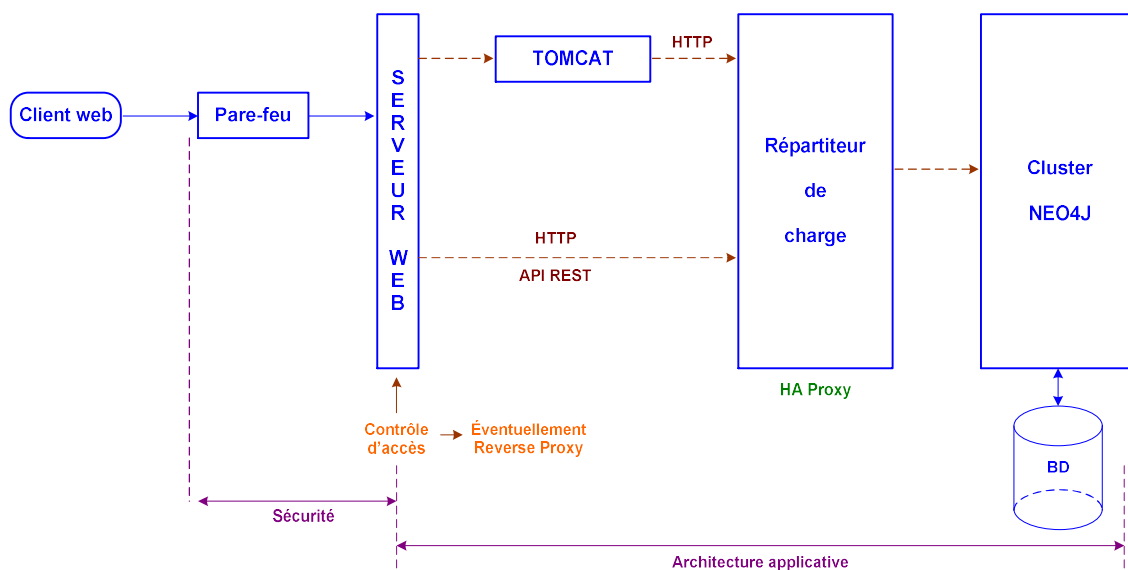
L'API REST avec le protocole http et l'échange de messages au format json, présente l'avantage d'être une solution standardisée.

BOLT :

Lorsque le volume de données à faire transiter par le réseau est important, il est plus efficace d'utiliser le protocole Bolt. Ce protocole basé sur TCP est plus rapide et plus compact que le protocole HTTP.

SÉCURITÉ DES ACCÈS

La sécurité est assez peu intégrée dans Neo4J. Neo4J est basé sur des API REST et tout le problème consiste donc à sécuriser ces API (accès HTTP). Il est possible de définir nativement des comptes et des mots de passe mais la granularité des accès reste basique. Il faut donc sécuriser les accès via des moyens externes à Neo4J.



SAUVEGARDES

Malgré les possibilités de réplication avec Neo4j, il est recommandé d'effectuer aussi des sauvegardes, plusieurs méthodes sont alors possibles :

- Méthode 1: copier manuellement les fichiers de la base :
`cp -R data/graph.db data/backup`
- Méthode 2: utiliser une commande de sauvegarde
`mkdir /.../backup/neo4j-backup`
`neo4j-backup -host 192.168.56.1 -to /.../backup/neo4j-backup`
- Méthode 3: utiliser la commande Cypher Dump
`neo4j-shell -c 'dump MATCH (n) OPTIONAL MATCH (n)-[r]-() RETURN r,n' > export.cql`

Pour restaurer le graphe (après avoir détruit ce qui existait auparavant):

`neo4j-shell -file export.cql`

LANGAGE CYPHER

Cypher est un langage déclaratif permettant d'interroger et mettre à jour le graphe. Ce langage est inspiré du SQL, on y retrouve beaucoup de concepts familiers, comme les clauses WHERE, ORDER BY, LIMIT...

Motifs

L'utilisateur définit des **motifs** qui sont recherchés dans tout le graphe. On utilise pour cela les labels (pour les nœuds) et les types (pour les relations).

Les nœuds :

Les nœuds sont représentés avec des parenthèses : `()`

Si vous avez besoin d'identifier le nœud dans votre requête (dans une clause WHERE par exemple), il suffit de lui donner un nom : `(monNoeud)`

Pour spécifier un label, il suffit de l'ajouter comme ceci : `(monNoeud:monLabel)`

Voici quelques exemples :

- `()` : n'importe quel nœud ;
- `(:Personne)` : un nœud avec le label Personne ;
- `(n:Personne)` : un nœud identifié dans la variable n avec le label Personne ;
- `(n:Personne:Acteur)` : un nœud identifié dans la variable n avec le label Personne et Acteur.

Les relations :

Les relations sont représentées par deux tirets et le signe supérieur : `-->`

Si vous avez besoin d'identifier la relation dans votre requête, vous pouvez lui donner un nom comme ceci :

- `[maRelation] ->`

Pour spécifier le type de la relation, il suffit de l'ajouter comme ceci : - `[maRelation:MON_TYPE] ->`

Exemple : `(moi)-[:AMI]->(mesAmis)-[:AMI]->(amisDeMesAmis)`

- `(a)--(b)` : n'importe quelle relation entre le nœud a et b (peu importe la direction) ;
- `(a)-[:AMI]->(b)` : relation de type AMI depuis le nœud a vers le nœud b ;
- `(a)-[r:AMICONNAIT]->(b)` : relation identifiée dans la variable r de type AMI ou CONNAIT depuis le nœud a vers le nœud b.

CRUD

Créer un nœud :

```
CREATE (neo:Company) //create node with label 'Company'
CREATE (neo:Company {name: 'Neo4j', hq: 'San Mateo'}) //create node with properties
```

Créer une relation :

```
CREATE (beginning_node)-[:edge_name{Attribute:'1', Attribute:'two'}]-(ending_node)
```

Effacer un nœud :

```
MATCH (n)
WHERE n.id=<monId>
DETACH DELETE n
```

Effacer les nœuds d'un label spécifique :

```
MATCH (n:Book)
DELETE n
```

Effacer les nœuds orphelins (sans lien) :

```
MATCH (n)
WHERE NOT (n)--()
DELETE n
```

Créer un groupe de liens :

```
Match (node_name:node_type {}), (node_name_two:node_type_two {})
CREATE (node_name)-[:edge_name{}]->(node_name_two)
```

Modifier un nœud :

```
MATCH (n)
WHERE n.some_attribute = "some identifier"
SET n.other_attribute = "a new value"
```


Rechercher un nœud d'après un label et la valeur d'un attribut:

```
MATCH (n:Book)
WHERE n.some_attribute = "some identifier"
RETURN n
```

Enchaîner des requêtes:

```
MATCH (n:Book)
WHERE n.title = "mon titre"
WITH n
```

```
MATCH (a:author)
WHERE a.name = n.author
RETURN a
```

Index

Par exemple pour créer un index sur le nom d'un acteur :

```
CREATE INDEX ON :Actor(name)
```

La plupart du temps un index est utilisé dans une requête

```
MATCH (actor:Actor { name: "Tom Hanks" })
RETURN actor;
```

Un index peut être multiple, c'est à dire composé de plusieurs propriétés, dans ce cas un nœud ne peut être ajouté que si il possède toutes les propriétés de l'index composite.

```
CREATE INDEX ON :Actor(name, born)
```

Il est possible d'afficher la liste des index définis dans une base de données en appelant la procédure stockée d'indexes :

```
CALL db.indexes
YIELD description, tokenNames, properties, type;
```

description	tokenNames	properties	type
"INDEX ON :Actor(name)"	["Actor"]	["name"]	"node_label_property"
"INDEX ON :Actor(name, born)"	["Actor"]	["name", "born"]	"node_label_property"

Contraintes

Les contraintes sont utilisées pour définir des règles sur les données du domaine (ex : unicité d'un nom).

Création d'une contraintes d'unicité du titre d'un film, cette contrainte ajoute implicitement un index sur la propriété.:

```
CREATE CONSTRAINT ON (movie:Movie) ASSERT movie.title IS UNIQUE
```

une contrainte peut être ajoutée sur une base qui contient des données.

Cela nécessite de mettre en conformité les données existantes.

Il est possible d'inspecter la base de données pour vérifier les contraintes qui sont définies :

```
procedure db.constraints:
CALL db.constraints
```

description
"CONSTRAINT ON (movie:Movie) ASSERT movie.title IS UNIQUE"

1 row

IMPORTATION DES DONNEES

Il existe plusieurs manières pour importer des données dans une base Neo4j :

- transactionnelle en effectuant des mises à jour en ligne : une liste d'instructions de création de nœuds et de relations.
- par lot (batch) pour de gros volumes à charger initialement: exécuter un script qui génère directement les fichiers de la base (import.sh), le serveur doit être inactif.
- Via une API Cypher (Load Csv) : à partir de n'importe quelle url ou en indiquant le chemin d'un fichier .csv
- Avec des extensions shell : certains contributeurs ont développé des outils dont l'exécution peut être déclenchée à distance à la manière de procédures stockées
- Dans une application avec un pilote Java: pour des besoins spécifiques une application peut être dédiée à l'importation de données.

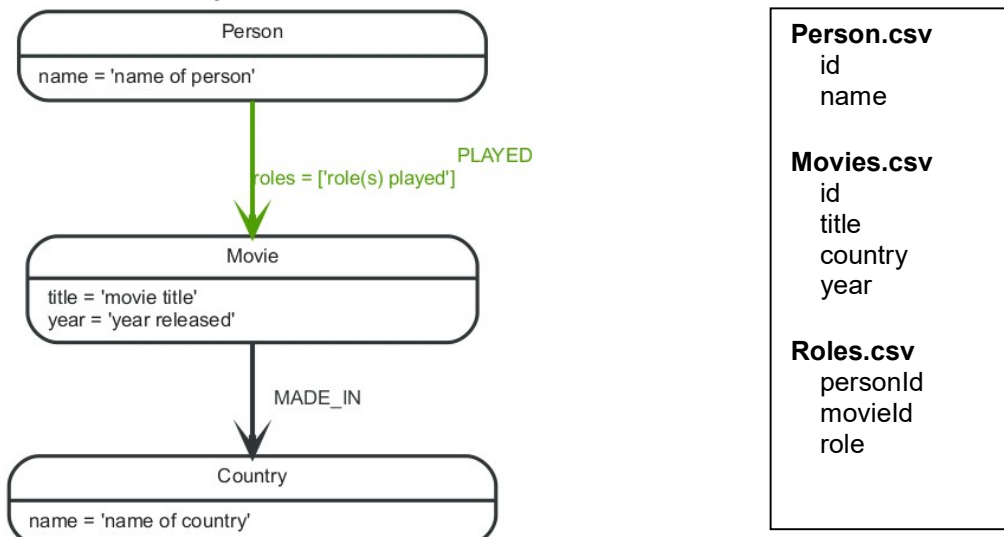
La démarche pour créer un graphe en important des données est la suivante :

- 1) la première étape et probablement la plus importante consiste à définir un schéma du graphe.
- 2) identifier la source de données, son format, sa taille.
- 3) définir le scénario d'importation, la séquence des opérations (création des index, des contraintes, ordre)
- 4) choisir un outil/méthode adapté

Ci-dessous les commandes Cypher pour l'importation de données au format .csv :

L'ETL propre au système Neo4j, permet d'importer facilement une grande quantité de données à partir de fichiers au format CSV. *Une ligne = les métadonnées d'un nœud ou d'une relation entre deux nœuds.*

Soit le schéma suivant et 3 fichiers csv.



Avant de réaliser l'import, il faut créer des index et des contraintes. C'est une protection face aux données éventuellement incorrectes dans les fichiers à importer.

```
CREATE CONSTRAINT ON (person:Person) ASSERT person.id IS UNIQUE
CREATE CONSTRAINT ON (movie:Movie) ASSERT movie.id IS UNIQUE
CREATE INDEX ON :Country(name)
```

Lorsque les données de plusieurs fichiers sont fusionnées, nous devons être certains d'avoir un index ou une contrainte d'unicité sur la propriété qui sert de lien.

Nous pouvons accéder directement aux fichiers sur le disque local ou via une URL.

La commande suivante crée un nœud pour chaque personne et pour chaque film ainsi qu'une relation entre les deux annotée d'un rôle. Nous conservons la traçabilité de la région où le film a été tourné :

```
LOAD CSV WITH HEADERS FROM "file:///persons.csv" AS csvLine
CREATE (p:Person {id: toInteger(csvLine.id), name: csvLine.name})

LOAD CSV WITH HEADERS FROM "file:///movies.csv" AS csvLine
MERGE (country:Country {name: csvLine.country})
CREATE (movie:Movie {id: toInteger(csvLine.id), title: csvLine.title, year:toInteger(csvLine.year)})
CREATE (movie)-[:MADE_IN]->(country)

USING PERIODIC COMMIT 500
LOAD CSV WITH HEADERS FROM "file:///roles.csv" AS csvLine
MATCH (person:Person {id: toInteger(csvLine.personId)}),(movie:Movie {id: toInteger(csvLine.movieId)})
CREATE (person)-[:PLAYED {role: csvLine.role}]->(movie)
```

Avec de nombreuses données, il est préférable d'utiliser la clause « USING PERIODIC COMMIT », pour indiquer à Neo4j de fractionner la transaction avec plusieurs commit périodiques.

MATCH définit une association entre deux champs. Pour chaque ligne du fichier csv qui associe les valeurs de ces deux champs, on crée un lien entre les nœuds correspondants.

MERGE effectue un regroupement et crée une collection de nœuds ayant pour identifiant la clé de regroupement.

Finalement comme la propriété id était seulement nécessaire pour importer les relations, il est possible d'effacer la contrainte et la propriété id de chaque nœud personne et film.

```
DROP CONSTRAINT ON (person:Person) ASSERT person.id IS UNIQUE
DROP CONSTRAINT ON (movie:Movie) ASSERT movie.id IS UNIQUE

MATCH (n)
WHERE n:Person OR n:Movie
REMOVE n.id
```

Il est possible de traiter plusieurs fois un même fichier csv pour modifier ou ajouter des propriétés aux nœuds et relations, par exemple suite à l'ajout d'une colonne dans ce fichier :

```
LOAD CSV WITH HEADERS FROM "file:///persons.csv" AS csvLine
MATCH (p:Person {id: toInteger(csvLine.id)})
SET p.name=T csvLine.name, p.firstname= csvLine.firstname
```

MONITORING

Neo4j dispose de plusieurs fichiers de journalisation des événements qui se produisent au sein d'une instance. Ces journaux (logging) contiennent des informations textuelles lisibles informant sur les traitements effectués par le système. Ces journaux sont de plusieurs types :

- debug.log : journal d'activité général de l'instance Neo4j ;
- gc.log : journal d'activité du Garbage Collector Java ;
- http.log : journal d'activité du serveur HTTP de l'instance Neo4j ;
- query.log : journalisation des requêtes CYPHER (édition entreprise uniquement) ;
- security.log : journalisation des événements liés à la sécurité (édition entreprise uniquement).

Cette journalisation est configurable. Afin de maîtriser notre espace disque, nous devons trouver un compromis entre le détail d'informations que nous avons besoin de consulter et l'espace disque que nous souhaitons dédier à cette journalisation. Seul le journal debug.log est activé par défaut.

PARTIE II : PRATIQUE

INSTALLATION

Docker : https://hub.docker.com/_/neo4j

How to use this image

You can start a Neo4j container like this:

```
docker run \  
--publish=7474:7474 --publish=7687:7687 \  
--volume=$HOME/neo4j/data:/data \  
neo4j
```

which allows you to access neo4j through your browser at <http://localhost:7474>.

This binds two ports (7474/7473 and 7687) for HTTP/HTTPS and Bolt access to the Neo4j API. A volume is bound to /data to allow the database to be persisted outside the container.

*By default, this requires you to login with neo4j/neo4j and change the password. You can, for development purposes, disable authentication by passing **--env=NEO4J_AUTH=none** to docker run.*

Démarrer le serveur Web Neo4j

- Dans le sous-répertoire **/bin** exécutez **./neo4j start**
- Dans un navigateur entrez l'url **<http://localhost:7474/>**
- Seulement la première fois, vous devrez vous connecter avec le compte par défaut et changer le mot de passe par défaut. À partir de la version 3.0.3 de la version community, le nom d'utilisateur et le mot de passe par défaut sont neo4j et neo4j.
- Vous pouvez maintenant insérer des requêtes Neo4j dans la console fournie par votre navigateur Web et examiner visuellement les résultats de chaque requête.

Créer une base de données Neo4j

Chaque serveur Neo4j (dans l'édition community) ne peut héberger qu'une seule base de données Neo4j. Au démarrage du serveur, si aucune base de données n'existe, Neo4j en crée une automatiquement. Pour installer une nouvelle base de données, vous devez :

- Dans le sous-répertoire **/bin** exécutez **./neo4j stop** pour arrêter le serveur.
- Dans le sous-répertoire **/conf** éditez le fichier **neo4j.conf** et modifiez la valeur du paramètre **dbms.active_database** qui contient le nom de la nouvelle base de données que vous souhaitez créer/utiliser.
- Dans le sous-répertoire **/bin** exécutez **./neo4j start** pour démarrer le serveur.
- Le serveur Web redémarre avec la nouvelle base de données vide. Vous pouvez lancer à nouveau **<http://localhost:7474/>** pour travailler avec cette base de données.
- La base de données créée se trouve dans le sous-répertoire **/data/databases**, sous un dossier portant le nom spécifié dans le paramètre **dbms.active_database**.

Supprimer l'une des bases de données

- Assurez-vous que le serveur Neo4j ne fonctionne pas. allez dans le sous-répertoire **/bin** et exécutez **./neo4j status**. Si le message indique que le serveur est en cours d'exécution, exécutez **./neo4j stop**.
- Ensuite, allez dans le sous-répertoire **/data/databases** et supprimez le dossier de la base de données que vous souhaitez recréer.

CLIENTS

Le terminal Neo4j permet d'exécuter des requêtes.

La console Neo4j est accessible sans aucune autre installation, il suffit d'exécuter. `/neo4j console` dans le sous répertoire `/bin`. Cet outil vous permet de créer en ligne une base Neo4j, de la requêter, mais surtout de la partager.

Le navigateur web: Neo4j est livré avec un serveur web, vous permettant de requêter et naviguer dans votre graphe. De nombreux exemples sont disponibles directement dans le browser (URL <http://localhost:7474/browser>)

Graphe gist (<http://graphgist.neo4j.com>) vous permet de réaliser une documentation de votre projet. Au lieu d'avoir des extraits de code figés, ici vos requêtes Cypher sont exécutées dans une console Neo4j, ce qui rend vos exemples dynamiques. Cet outil est beaucoup utilisé pour rédiger des cas d'utilisation : on y définit une problématique et l'on y démontre comment Neo4j peut répondre via une série de requêtes Cypher. Exemples de gist – cas d'utilisation: <http://graphgist.neo4j.com/#!/gists/all>

Outil graphe : GEPHI permet d'importer et d'exporter des graphes au format Neo4j.

JEU DE DONNEES

Pour réaliser mes essais j'ai utilisé deux jeux de données Opendata qui peuvent être corrélés:

- le premier jeu de données concerne les entreprises:
Les données sont accessibles sur le site : <https://opendata.datainfogreffe.fr/page/offre-open-data/>

Liste des sociétés commerciales immatriculées au registre du commerce et des sociétés en 2020.
Mots clés : immatriculation, création, sociétés, entreprises

Les chiffres clés des sociétés commerciales ayant déposé leurs comptes annuels pour l'exercice 2019, enrichis des années 2017 et 2018
Mots clés: chiffre d'affaires, comptes annuels, effectif, résultat d'exercice

Le téléchargement des données est possible au format csv et Json.

- le deuxième jeu de données est la Base Adresse Nationale BAN
Ce jeu de données provient d'un service public certifié (données de référence publiques)

La Base Adresse Nationale est une base de données qui a pour but de référencer l'intégralité des adresses du territoire français. Elle contient la position géographique de plus de 24 millions d'adresses.

Elle est constituée par la collaboration entre Etalab, l'IGN et les collectivités locales, avec la contribution de la DGFIP, de l'INSEE, de La Poste et de l'ARCEP.

Elle est diffusée sur le site adresse.data.gouv.fr développé par la mission Etalab de la direction interministérielle du numérique (DINUM).

Les données sont disponibles sous Licence Ouverte.

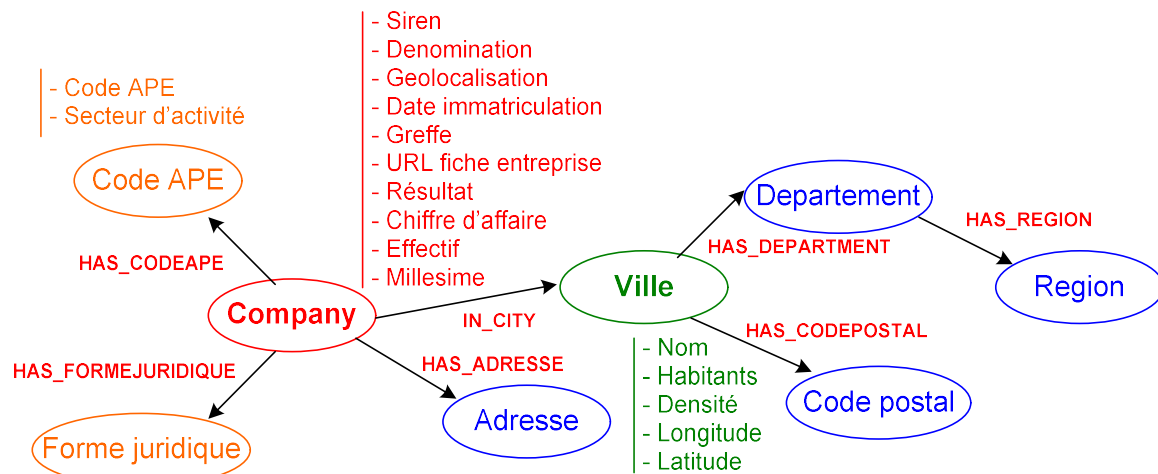
Un service de géocodage gratuit est mis à disposition par la mission Etalab.

Le téléchargement des données est possible au format csv et Json.

1 ligne par adresse
24,6 millions d'adresses
200 000 lieux-dits (beta)
Contient les libellés normalisés AFNOR

MODELE DE DONNEES

Bien qu'il ne soit pas imposé de définir un schéma avec le système Neo4j, cela constitue tout de même la première étape, très importante. Cette structuration des informations permet de définir l'endroit où les données sont rangées, c'est une vue qu'il faut bien connaître afin de construire les requêtes.



IMPORTATION DE DONNEES

Ces requêtes peuvent être saisies et exécutées via l'interface web. Il est important de lancer la création des index avant d'importer les données.

```
// Nœuds : Company - 231331 sociétés
CREATE INDEX ON :Company(CodeSiren) // ou: CREATE INDEX FOR (Company:Label) ON (Company.CodeSiren)
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
CREATE (e:Company {CodeSiren:lines.Siren, Denomination:lines.Denomination,
Geolocalisation:lines.Geolocalisation, DateImmatriculation:lines.Date_immatriculation})

// Nœuds : FormeJuridique -- 97 formes juridiques
CREATE INDEX ON :FormeJuridique(name)
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MERGE (fj:FormeJuridique {name: lines.FormeJuridique})

// Nœuds : CodeAPE -- 798 APE
CREATE INDEX ON :CodeAPE(CodeAPE)
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MERGE (ca:CodeAPE {CodeAPE: lines.CodeAPE, LibelleAPE: lines.LibelleAPE})

// Nœuds : Adresse -- 171685 adresses
CREATE INDEX ON :Adresse(name)
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MERGE (adr:Adresse {name: lines.Adresse})

// Nœuds : CodePostal -- 6537 code postal
CREATE INDEX ON :CodePostal(name)
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MERGE (cdp:CodePostal {name: lines.CodePostal})

// Nœuds : Ville -- 22272 villes
CREATE INDEX ON :Ville(name)
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MERGE (vi:Ville {name: lines.Ville})

// Nœuds : Region -- 16 regions
CREATE INDEX ON :Region(name)
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MERGE (vi:Region {name: lines.Region})
```

```

// Liaisons : Adresse <-> CodePostal -- 197975
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (adresse:Adresse {name:lines.Adresse}), (codepostal:CodePostal {name:lines.CodePostal})
CREATE UNIQUE (adresse)-[:HAS_CODEPOSTAL]->(codepostal)

// Liaisons : CodePostal <-> Ville -- 23854
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (city:Ville {name:lines.Ville}), (codepostal:CodePostal {name:lines.CodePostal})
CREATE UNIQUE (codepostal)-[:HAS_CITY]->(city)

// Liaisons : Ville <-> Region -- 23015
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (region:Region {name:lines.Region}), (ville:Ville {name:lines.Ville})
CREATE UNIQUE (ville)-[:HAS_REGION]->(region)

// Liaisons : Company <-> CodeAPE -- 234711
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (cny:Company {CodeSiren:lines.Siren}), (ape:CodeAPE {CodeAPE:lines.CodeAPE})
CREATE UNIQUE (cny)-[:HAS_CODEAPE]->(ape)

// Liaisons : Company <-> FormeJuridique -- 231360
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (cny:Company {CodeSiren:lines.Siren}), (fj:FormeJuridique {name:lines.FormeJuridique})
CREATE UNIQUE (cny)-[:HAS_FORMEJURIDIQUE]->(fj)

// Liaisons : Company <-> Adresse -- 231405
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (cny:Company {CodeSiren:lines.Siren}), (adr:Adresse {name:lines.Adresse})
CREATE UNIQUE (cny)-[:HAS_ADRESSE]->(adr)

// Ajout/modif de propriétés des noeuds : Company -- Set 375779 properties
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (c:Company {CodeSiren:lines.Siren})
SET c.Resultat=ToInt(lines.Resultat1), c.CA=ToInt(lines.CA1), c.Effectif=ToInt(lines.Effectif1),
c.Millesime=lines.Millesime1

// Ajout/modif de propriétés des noeuds : Ville
LOAD CSV WITH HEADERS FROM "file:///villes.csv" AS lines
MATCH (v:Ville {name:lines.Ville})
SET v.CodeInsee=lines.Code_INSEE, v.latitude=toFloat(lines.Latitude), v.longitude=toFloat(lines.Longitude),
v.Habitants=toInt(lines.Habitants_1990), v.Densite=toInt(lines.Densite)

// Nœud : Départements --- Added 96 labels, created 96 nodes, set 192 properties, completed after 3636 ms.
CREATE INDEX ON :Departement(code)
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
WITH lines WHERE NOT lines.Departement IS null
MERGE (d:Departement {code:lines.NumDept, name:lines.Departement})

// Liaisons : Departement <-> Region -- Created 102 relationships, completed after 6959 ms.
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (region:Region {name:lines.Region}), (dpt:Departement {code:lines.NumDept})
CREATE UNIQUE (dpt)-[:HAS_REGION]->(region)

// Liaisons : Company <-> Ville -- Created 231439 relationships, completed after 12246 ms.
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (city:Ville {name:lines.Ville}), (cny:Company {CodeSiren:lines.Siren})
CREATE UNIQUE (cny)-[:IN_CITY]->(city)

// Liaisons : Ville <-> Departement -- Created 23082 relationships, completed after 5793 ms.
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (dpt:Departement {code:lines.NumDept}), (city:Ville {name:lines.Ville})
CREATE UNIQUE (city)-[:HAS_DEPARTMENT]->(dpt)

// Liaisons : Ville <-> CodePostal ---- Created 23854 relationships, completed after 7054 ms.
LOAD CSV WITH HEADERS FROM "file:///chiffres-cles-2019_v.csv" AS lines
MATCH (city:Ville {name:lines.Ville}), (codepostal:CodePostal {name:lines.CodePostal})
CREATE UNIQUE (city)-[:HAS_CODEPOSTAL]->(codepostal)

-----
// POUR VERIFIER
MATCH (a:CodeAPE {LibelleAPE:"Restauration traditionnelle"})<-[:HAS_CODEAPE]-(b:Company)-[*]->(c:CodePostal)
WITH count(b) as score, c
RETURN c.name, score
ORDER BY score DESC
LIMIT 5

```

INTERROGATION DE LA BASE DE DONNEES

```
// retourner un graphe
MATCH (n1)-[r]->(n2) RETURN r, n1, n2 LIMIT 100

// nombre de régions
MATCH (n:Region) RETURN count(*) as count

// nombre de liens Region-Ville
MATCH p=()-[r:HAS_REGION]->() RETURN count(p) as count
=> 23015

// les noms de villes associées à une région
MATCH p=(v:Ville)-[l:HAS_REGION]->(r:Region) WHERE r.name='Corse' RETURN v.name
MATCH p=(v:Ville)-[l:HAS_REGION]->(r:Region) WHERE r.name='Corse' RETURN count(v.name)
=> 113

// nombre d'adresses indéterminées
MATCH (cny:Company)-[:HAS_ADRESSE]->(adr:Adresse) WHERE adr.name='INDETERMINEE' RETURN count(cny)
==> 2023

// retourne 25 liens entre villes et régions --- COPIER LE GRAPHE ---
MATCH p=()-[r:HAS_REGION]->() RETURN p LIMIT 25

// Les company du type "Programmation informatique"
MATCH p=(cny:Company)-[r:HAS_CODEAPE]->(cod:CodeAPE) WHERE cod.LibelleAPE='Programmation informatique'
RETURN cny.Denomination
==> 1561

    cny.Denomination
    "Gotta Apps"
    "WEB 14"
    "SAS KASO"
    "La Gaviota"
    "IBSYSTEMS IT CONSULTING"
    "IIEESS"
    "VENTO TECHNOLOGIE"
    "WEB KONSEIL"
    "NUIT ET SPA"
    ...

// Société dont le chiffre d'affaire est supérieur à 1 Million
MATCH (c:Company)
WHERE c.CA > 1000000
return c.Denomination,c.CA, c.Effectif ORDER BY c.CA DESC

MATCH (c:Company)
WHERE c.CA > 1000000
return count(c)
==> 18129

    c.Denomination          c.CA          c.Effectif
    "RMB"                   78000116080   null
    "ALLIANCE HEALTHCARE REPARTITION" 3540203255   2209
    "MEUBLES IKEA FRANCE"    3006553751   10099
    "ALSTOM TRANSPORT SA"    2981000000   null
    "TOYOTA MOTOR MANUFACTURING FRANCE" 2879848076   null
    "CASTORAMA FRANCE"       2603295673   13278
    "BRICO DEPOT"            2323998686   null
    "NISSAN WEST EUROPE"     2182193000   269

// Société dont le résultat est supérieur à 30000€
MATCH (c:Company)
WHERE c.Resultat > 30000
return c.Denomination,c.Resultat ORDER BY c.Resultat DESC

    c.Denomination          c.Resultat
    "ALSTOM HOLDINGS"       7290000000
    "EUTELSAT S A"          5640000000
    "TAT"                    233058616
    "KINGFISHER INVESTISSEMENTS" 204661834
    "UBISOFT ENTERTAINMENT" 159159000
    "ORFIM"                  127981465
    "REMY COINTREAU SERVICES" 127037598
    "Soitec"                 108460000
    "RICHEMONT HOLDING FRANCE" 107465320
```



```
// Adresses dans le gard
MATCH p=(adresse)-[:HAS_CODEPOSTAL]->(cp:CodePostal) WHERE cp.name='30000' return p

// les company d'une ville
MATCH (n1:Company)-[r:IN_CITY]->(n2:Ville) RETURN r, n1, n2 LIMIT 100
MATCH (n1:Company)-[r:IN_CITY]->(n2:Ville) WHERE n2.name='NIMES' RETURN r, n1, n2 LIMIT 100

// Nombre de société "Editeur de logiciels applicatifs"
MATCH (a:CodeAPE {LibelleAPE:"Edition de logiciels applicatifs"})<-[:HAS_CODEAPE]-(b:Company)
RETURN count(b)
==> 326

// Les 10 premières sociétés "Editeur de logiciels applicatifs" par ordre décroissant de leur chiffre d'affaire
MATCH (a:CodeAPE {LibelleAPE:"Edition de logiciels applicatifs"})<-[:HAS_CODEAPE]-(b:Company)
WHERE b.CA IS NOT NULL
WITH b.CA as score, b
RETURN b.Denomination, b.CA
ORDER BY score DESC
LIMIT 10

      b.Denomination      b.CA
"ORACLE FRANCE"          941379286
"MUREX"                   341891578
"OPEN TEXT"               98075100
"ISAGRI"                  84224401
"GENERIX GROUP"           56930000
"JVS-MAIRISTEM"          28678242
"ARTURIA"                 22968600
"AVANQUEST SOFTWARE"     21721601
"DYNATRACE"               21547575
"SALVIA DEVELOPPEMENT" 15883668

// sociétés "Editeur de logiciels applicatifs" dans le département 84 (vaucluse)
MATCH (a:CodeAPE {LibelleAPE:"Edition de logiciels applicatifs"})<-[:HAS_CODEAPE]-(b:Company)-[:IN_CITY]-
>(v:Ville)-[:HAS_DEPARTMENT]-(d:Departement {code:"84"})
RETURN b.Denomination, b.CA

      b.Denomination      b.CA
"ASSISTANCE ET PROGRAMMATION INFORMATIQUES DU COMTAT" null
"GREET"                  null
"BOURSE PREMIUM PROFESSIONNEL" null
"SPRING"                  74000

// 10 sociétés "Editeur de logiciels applicatifs" autour de Nîmes situées à une distance inférieure à 300000
- les plus éloignées
MATCH (a:CodeAPE {LibelleAPE:"Edition de logiciels applicatifs"})<-[:HAS_CODEAPE]-(c:Company)-[:IN_CITY]-
>(v:Ville)
WITH point({ longitude: v.longitude, latitude: v.latitude }) AS location, point({longitude:4.35,
latitude:43.833333}) as nimesLocation,v,c
WHERE location IS NOT null
WITH round(distance(location, nimesLocation)) AS nimesDistance, v,c
WHERE nimesDistance < 300000
RETURN c.Denomination, v.name, nimesDistance
ORDER BY nimesDistance DESC
LIMIT 10

      c.Denomination      v.name      nimesDistance
"ABDR"                   ST PRIEST"  296574.0
"MICGEST SOFTWARE"       "BELLIGNAT"  286869.0
"CAP ON LINE"             "BUELLAS"   270567.0
"Vega QuatuHoRe IDF"     "AUCAMVILLE"  251693.0
"MD DYNAMICS"            "BRAX"      250534.0
"SUPER LOGICIELS SARL"   "VILLENEUVE TOLOSANE" 245414.0
"Agoora"                  "MISERIEUX"  242147.0
"EnerBIM"                 "SEILH"     241615.0
"LEARNPULSE"             "TOULOUSE"  236090.0
"AGX INFORMATIQUE"       "TOULOUSE"  236090.0
```

```

// les plus proches
RETURN c.Denomination, v.name, nimesDistance
ORDER BY nimesDistance ASC
LIMIT 10

c.Denomination      v.name      nimesDistance
"CAPPSOFT"          "VAUVERT"   15781.0
"HEXTRAKT"          "LUNEL"     24131.0
"CTI SOFTWARE"      "ARLES"     29382.0
"MOVE IN MED"       "BAILLARGUES" 32599.0
"POWER GRID SCADA"  "MONTPELLIER" 45656.0
"Wacky Gamer Tools" "MONTPELLIER" 45656.0
"INFORMATIQUE SERVICE" "FABREGUES" 56563.0
"LGP SOLUTIONS"     "SENAS"     59654.0
"FAAROO"            "VILLENEUVE DE BERG" 80672.0
"FINOPSYS"         "AIX EN PROVENCE" 93386.0

// nombre
RETURN count(c)
==> 59

// cherche une adresse géographique en utilisant une procédure APOC et le service OpenStreetMap.
CALL apoc.spatial.geocode('Nîmes')
YIELD location
RETURN location as p
LIMIT 1

{
  "description": "Nîmes, Gard, Occitanie, France métropolitaine, France",
  "latitude": 43.8374249,
  "longitude": 4.3600687
}

// les 10 sociétés les plus proches de Nîmes dans le domaine "Edition de logiciels applicatifs"
// en utilisant une procédure stockée APOC et le service OpenStreetMap.
CALL apoc.spatial.geocode('Nîmes')
YIELD location
WITH location as p
LIMIT 1

MATCH (a:CodeAPE {LibelleAPE:"Edition de logiciels applicatifs"})<-[:HAS_CODEAPE]-(c:Company)-[:IN_CITY]->(v:Ville)
WITH point({ longitude: v.longitude, latitude: v.latitude }) AS location, point({longitude:p.longitude, latitude:p.latitude}) as nimesLocation,v,c
WHERE location IS NOT null
WITH round(distance(location, nimesLocation)) AS nimesDistance, v,c
WHERE nimesDistance < 300000
RETURN c.Denomination, v.name, nimesDistance
ORDER BY nimesDistance ASC
LIMIT 10

c.Denomination      v.name      nimesDistance
"CAPPSOFT"          "VAUVERT"   16495.0
"HEXTRAKT"          "LUNEL"     25031.0
"CTI SOFTWARE"      "ARLES"     29054.0
"MOVE IN MED"       "BAILLARGUES" 33523.0
"POWER GRID SCADA"  "MONTPELLIER" 46580.0
"Wacky Gamer Tools" "MONTPELLIER" 46580.0
"INFORMATIQUE SERVICE" "FABREGUES" 57489.0
"LGP SOLUTIONS"     "SENAS"     58927.0
"FAAROO"            "VILLENEUVE DE BERG" 80106.0
"FINOPSYS"         "AIX EN PROVENCE" 92792.0

// pour retourner le graphe (voir annexe)
MATCH r=(a:CodeAPE {LibelleAPE:"Edition de logiciels applicatifs"})<-[:HAS_CODEAPE]-(c:Company)-[:IN_CITY]->(v:Ville)
WITH point({ longitude: v.longitude, latitude: v.latitude }) AS location, point({longitude:p.longitude, latitude:p.latitude}) as nimesLocation,v,c,r
WHERE location IS NOT null
WITH round(distance(location, nimesLocation)) AS nimesDistance,r
WHERE nimesDistance < 300000
RETURN r
ORDER BY nimesDistance ASC
LIMIT 10

```

1) Configuration du cluster causal

Installation de trois serveurs Neo4j (version entreprise) de type core et un serveur du type replica sur quatre conteneurs docker.

Rappel : les paramètres de configuration d'une instance Neo4j se définissent dans le fichier de configuration neo4j.conf. Cependant, lors de l'instanciation d'un conteneur docker, il est possible de transmettre les valeurs de ces paramètres. Ci-dessous, les paramètres de configuration d'un cluster causal :

dbms.mode : indique le type d'installation du serveur Neo4j. Les valeurs possibles sont CORE ou READ_REPLICA.

causal_clustering.expected_core_cluster_size : il s'agit du nombre d'instances core attendues pour former le cluster core en fonction de la règle : $M=2F+1$ où M correspond au nombre total de serveurs core nécessaires pour absorber F défaillances.

causal_clustering.initial_discovery_members : liste des adresses des membres du cluster core. L'adresse d'un serveur est constituée de son IP (ou le nom d'hôte) suivie du port 5000. Il est possible de ne lister que l'instance en cours de paramétrage (si c'est un serveur core), les autres instances seront alors découvertes par le protocole de découverte.

Les paramètres suivants concernent les connecteurs :

dbms_connector_bolt_listen_address : adresse IP et numéro de port bolt

dbms_connector_http_listen_address : adresse IP et numéro de port http

dbms_connectors_default_advertised_address : adresse par défaut que les clients devraient utiliser pour se connecter à cette instance.

Par défaut l'adresse IP est égale à 0.0.0.0 ce qui permet à l'instance d'écouter les messages arrivant sur n'importe quelle interface réseau.

Attention : Il ne faut pas d'espaces avant ni après le symbole = du paramètre.

```
// Création du réseau nommé : cluster
docker network create --driver=bridge cluster

// core 1
sudo docker run --name=core1 --detach --network=cluster \
--publish=7474:7474 \
--publish=7687:7687 \
--env=NEO4J_dbms_mode= CORE \
--env=NEO4J_causalClustering_expectedCoreClusterSize=3 \
--env=NEO4J_causalClustering_initialDiscoveryMembers=core1:5000,core2:5000,core3:5000 \
--env=NEO4J_AUTH=none \
--env=NEO4J_dbms_connectors_default_advertised_address= \
--env=NEO4J_dbms_connector_bolt_listen_address=:7687 \
--env=NEO4J_dbms_connector_http_listen_address=:7474 \
neo4j:3.2.2-enterprise
// core 2
sudo docker run --name=core2 --detach --network=cluster \
--publish=7475:7475 \
--publish=7688:7688 \
--env=NEO4J_dbms_mode= CORE \
--env=NEO4J_causalClustering_expectedCoreClusterSize=3 \
--env=NEO4J_causalClustering_initialDiscoveryMembers=core1:5000,core2:5000,core3:5000 \
--env=NEO4J_AUTH=none \
--env=NEO4J_dbms_connectors_default_advertised_address= \
--env=NEO4J_dbms_connector_bolt_listen_address=:7688 \
--env=NEO4J_dbms_connector_http_listen_address=:7475 \
neo4j:3.2.2-enterprise
// core 3
sudo docker run --name=core3 --detach --network=cluster \
--publish=7476:7476 \
--publish=7689:7689 \
--env=NEO4J_dbms_mode= CORE \
--env=NEO4J_causalClustering_expectedCoreClusterSize=3 \
--env=NEO4J_causalClustering_initialDiscoveryMembers=core1:5000,core2:5000,core3:5000 \
--env=NEO4J_AUTH=none \
--env=NEO4J_dbms_connectors_default_advertised_address= \
--env=NEO4J_dbms_connector_bolt_listen_address=:7689 \
--env=NEO4J_dbms_connector_http_listen_address=:7476 \
neo4j:3.2.2-enterprise
```

```
// replica 1
sudo docker run --name=replica1 --detach --network=cluster \
--publish=7484:7484 \
--publish=7697:7697 \
--env=NEO4J_dbms_mode=READ_REPLICA \
--env=NEO4J_causalClustering_expectedCoreClusterSize=3 \
--env=NEO4J_causalClustering_initialDiscoveryMembers=core1:5000,core2:5000,core3:5000 \
--env=NEO4J_AUTH=none \
--env=NEO4J_dbms_connectors_default__advertised__address= \
--env=NEO4J_dbms_connector_bolt_listen__address=:7697 \
--env=NEO4J_dbms_connector_http_listen__address=:7484 \
neo4j:3.2.2-enterprise
```

Nous pouvons suivre l'état du cluster depuis un navigateur (<http://localhost:7474/>), en appelant la procédure : **CALL dbms.cluster.overview()** :

\$ CALL dbms.cluster.overview()

	id	addresses	role	groups
Table	"1a4bb621-4183-419c-8add-11df7bcd192d"	["bolt://localhost:7687", "http://localhost:7474", "https://localhost:7473"]	"LEADER"	[]
Text	"d8a5cdc0-6776-4fc2-8020-630f84f58496"	["bolt://localhost:7688", "http://localhost:7475", "https://localhost:7473"]	"FOLLOWER"	[]
	"f7e6edcc-2a5f-4422-b6d3-aa3cc3ac4622"	["bolt://localhost:7689", "http://localhost:7476", "https://localhost:7473"]	"FOLLOWER"	[]
Code	"a42937c6-eb2d-4b6f-8720-539b2911bce5"	["bolt://localhost:7697", "http://localhost:7484", "https://localhost:7473"]	"READ_REPLICA"	[]

L'état de ces quatre instances docker peut être affiché dans un terminal avec la commande :
\$ docker ps

CONTAINER ID	IMAGE	PORTS	NAMES
1bc90bf4f1be	neo4j:3.2.2-enterprise	7473-7474/tcp, 0.0.0.0:7484->7484/tcp, 7687/tcp, 0.0.0.0:7697->7697/tcp	replica1
1d98cea83dca	neo4j:3.2.2-enterprise	7473-7474/tcp, 0.0.0.0:7476->7476/tcp, 7687/tcp, 0.0.0.0:7689->7689/tcp	core3
f03b0e806e45	neo4j:3.2.2-enterprise	7473-7474/tcp, 0.0.0.0:7475->7475/tcp, 7687/tcp, 0.0.0.0:7688->7688/tcp	core2
ac060ea4f5a1	neo4j:3.2.2-enterprise	0.0.0.0:7474->7474/tcp, 7473/tcp, 0.0.0.0:7687->7687/tcp	core1

2) Import des données

Copier le fichier csv dans l'instance docker

\$ sudo docker cp ./villes.csv ac060ea4f5a1:/var/lib/neo4j/import/villes.csv

```
// villes
CREATE INDEX ON :Ville(name)

LOAD CSV WITH HEADERS FROM "file:///villes.csv" AS lines
MERGE (v:Ville {name:lines.Ville})

LOAD CSV WITH HEADERS FROM "file:///villes.csv" AS lines
MATCH (v:Ville {name:lines.Ville})
SET v.CodeInsee=lines.Code_INSEE,v.latitude=toFloat(lines.Longitude),
v.longitude=toFloat(lines.Longitude),
v.Habitants=toInt(lines.Habitants_1990),v.Densite=toInt(lines.Densite)
```

Lecture des données sur n'importe quel nœud : (<http://localhost:7474/>, [7475/](http://localhost:7475/), [7476/](http://localhost:7476/), [7484/](http://localhost:7484/))

```
//lecture des 10 villes dont la densité est la plus élevée
MATCH (v:Ville)
WHERE v.Densite IS NOT NULL
RETURN v.name,v.Densite
ORDER BY v.Densite DESC
LIMIT 10
```

v.name	v.Densite
"GRENOBLE"	8315
"LE CANNET"	5427
"STE FOY LA GRANDE"	5382
"NICE"	4761
"VICHY"	4737
"LES NOES PRES TROYES"	4655
"TROYES"	4489
"BEAUSOLEIL"	4418
"CAEN"	4391
"ECHIROLLES"	4381

3) Arrêt du Leader (Maître)

via kitematic ou la commande neo4j stop

Etat du cluster depuis un navigateur (<http://localhost:7475/>): **CALL dbms.cluster.overview()**

```
$ CALL dbms.cluster.overview()
```

	id	addresses	role
Table	"d8a5cdc0-6776-4fc2-8020-630f84f58496"	["bolt://localhost:7688", "http://localhost:7475", "https://localhost:7473"]	"LEADER"
Text	"f7e6edcc-2a5f-4422-b6d3-aa3cc3ac4622"	["bolt://localhost:7689", "http://localhost:7476", "https://localhost:7473"]	"FOLLOWER"
	"a42937c6-eb2d-4b6f-8720-539b2911bce5"	["bolt://localhost:7697", "http://localhost:7484", "https://localhost:7473"]	"READ_REPLICA"

On peut constater que le serveur 2 a été élu Leader

4) suppression dans le graphe du nœud GRENOBLE puis remise en service serveur 1

```
// suppression du nœud ="GRENOBLE"  
MATCH (v:Ville)  
WHERE v.name="GRENOBLE"  
DELETE v
```

On constate que la suppression ne peut être réalisée que sur le Leader :

```
$ MATCH (v:Ville) WHERE v.name="LE CANNET" DELETE v
```



ERROR Neo.ClientError.Cluster.NotALeader

No write operations are allowed directly on this database. Writes must pass through the leader. The role of this server is: FOLLOWER

le serveur 1 est devenu Follower.

```
$ CALL dbms.cluster.overview()
```



	id	addresses	role	groups
Table	"1a4bb621-4183-419c-8add-11dfbcd192d"	["bolt://localhost:7687", "http://localhost:7474", "https://localhost:7473"]	"FOLLOWER"	[]
Text	"d8a5cdc0-6776-4fc2-8020-630f84f58496"	["bolt://localhost:7688", "http://localhost:7475", "https://localhost:7473"]	"LEADER"	[]
	"f7e6edcc-2a5f-4422-b6d3-aa3cc3ac4622"	["bolt://localhost:7689", "http://localhost:7476", "https://localhost:7473"]	"FOLLOWER"	[]
Code	"a42937c6-eb2d-4b6f-8720-539b2911bce5"	["bolt://localhost:7697", "http://localhost:7484", "https://localhost:7473"]	"READ_REPLICA"	[]

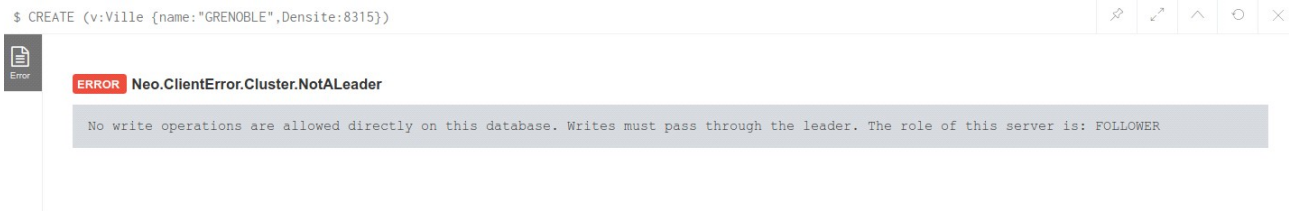
Le serveur 1 s'est bien synchronisé :

```
$ MATCH (v:Ville) WHERE v.Densite IS NOT NULL RETURN v.name,v.Densite ORDER BY v.Densite DESC LIMIT 10
```

	v.name	v.Densite
Table	"LE CANNET"	5427
Text	"STE FOY LA GRANDE"	5382
	"NICE"	4761
Code	"VICHY"	4737
	"LES NOES PRES TROYES"	4655
	"TROYES"	4489
	"BEAUSOLEIL"	4418
	"CAEN"	4391
	"ECHIROLLES"	4381
	"BORDEAUX"	4261

5) Arrêt du Leader et observation du journal d'activité de l'instance Neo4j

On constate que la creation d'un nœud dans le graphe ne peut être réalisée que sur le Leader :



Suite à l'arrêt du serveur 2 (Leader) le serveur 1 est élu :

```
$ CALL dbms.cluster.overview()
```

	id	addresses	role
Table	"1a4bb621-4183-419c-8add-11d7bcd192d"	["bolt://localhost:7687", "http://localhost:7474", "https://localhost:7473"]	"LEA"
Text	"f7e6edcc-2a5f-4422-b6d3-aa3cc3ac4622"	["bolt://localhost:7689", "http://localhost:7476", "https://localhost:7473"]	"FOL"
	"a42937c6-eb2d-4b6f-8720-539b2911bce5"	["bolt://localhost:7697", "http://localhost:7484", "https://localhost:7473"]	"REA"

Information sur le connecteur :

```
2020-05-21 08:37:44.527+0000 INFO [o.n.c.d.HazelcastCoreTopologyService] My connection info: [
  Discovery: listen=0.0.0.0:5000, advertised=ac060ea4f5a1:5000,
  Transaction: listen=0.0.0.0:6000, advertised=ac060ea4f5a1:6000,
  Raft: listen=0.0.0.0:7000, advertised=ac060ea4f5a1:7000,
  Client Connector Addresses: bolt://localhost:7687,http://localhost:7474,https://localhost:7473
]
```

Sur évènement « suppression d'un membre core », une élection est enclenchée, le nouveau leader est élu au bout de 8 secondes :

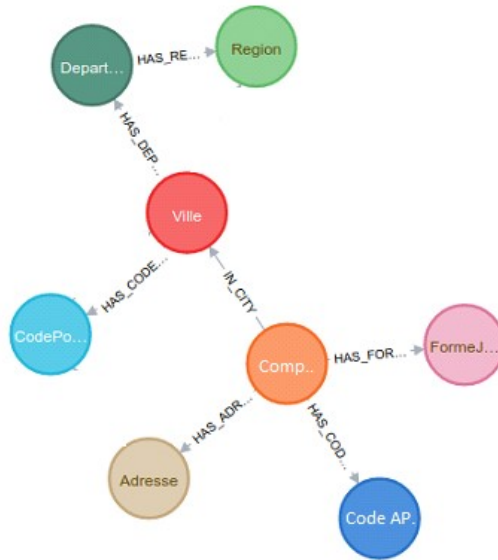
```
2020-05-21 10:03:51.653+0000 INFO [o.n.c.d.HazelcastCoreTopologyService] Core member removed MembershipEvent
{member=Member [core2]:5000 - 86ac2720-e1a2-4c24-ae5c-6b402733a794,type=removed}
2020-05-21 10:03:52.054+0000 INFO [o.n.c.d.HazelcastCoreTopologyService] Core topology changed {added=[],
removed=[{memberId=MemberId{d8a5cdc0}, info=CoreServerInfo{raftServer=f03b0e806e45:7000,
catchupServer=f03b0e806e45:6000,
clientConnectorAddresses=bolt://localhost:7688,http://localhost:7475,https://localhost:7473, groups=[]}}]}
2020-05-21 10:03:52.054+0000 INFO [o.n.c.c.c.m.RaftMembershipManager] Target membership:
[MemberId{f7e6edcc}, MemberId{1a4bb621}]
2020-05-21 10:03:52.054+0000 INFO [o.n.c.c.c.m.RaftMembershipManager] Not safe to remove member
[MemberId{d8a5cdc0}] because it would reduce the number of voting members below the expected cluster size of
3. Voting members: [MemberId{1a4bb621}, MemberId{f7e6edcc}, MemberId{d8a5cdc0}]

2020-05-21 10:03:59.573+0000 INFO [o.n.c.c.c.RaftMachine] Election timeout triggered
2020-05-21 10:03:59.574+0000 INFO [o.n.c.c.c.RaftMachine] Election started with vote request: Vote.Request
from MemberId{1a4bb621} {term=3, candidate=MemberId{1a4bb621}, lastAppended=176, lastLogTerm=2} and members:
[MemberId{1a4bb621}, MemberId{f7e6edcc}, MemberId{d8a5cdc0}]

2020-05-21 10:03:59.575+0000 INFO [o.n.c.c.c.RaftMachine] Moving to CANDIDATE state after successfully
starting election
2020-05-21 10:03:59.741+0000 INFO [o.n.c.m.SenderService] Creating channel to: [1d98cea83dca:7000]
2020-05-21 10:03:59.741+0000 INFO [o.n.c.m.RaftOutbound] No address found for MemberId{d8a5cdc0}, probably
because the member has been shut down.
2020-05-21 10:03:59.745+0000 INFO [o.n.c.m.SenderService] Connected: [id: 0x32325eb1, L:/172.19.0.2:55158 -
R:1d98cea83dca/172.19.0.4:7000]
2020-05-21 10:03:59.918+0000 INFO [o.n.c.c.c.RaftMachine] Moving to LEADER state at term 3 (I am
MemberId{1a4bb621}), voted for by [MemberId{f7e6edcc}]
2020-05-21 10:03:59.920+0000 INFO [o.n.c.c.c.s.RaftState] Leader changed from MemberId{d8a5cdc0} to
MemberId{1a4bb621}
```

ANNEXES

Db . schema



Dix sociétés "Editeur de logiciels applicatifs" autour de Nîmes situées à une distance inférieure à 300000 - les plus proches

